

DOML-F: Uma Linguagem Declarativa e Modelo Observacional Auditável para Replicação e Experimentação Forense em Ambientes Linux

DOML-F: A Declarative Language and Auditable Observational Model for Forensic Replication and Experimentation in Linux Environments

Marconi Ferreira de Medeiros

Universidade Federal de São João del-Rei (UFSJ)
CEP 36307-352 – São João del-Rei – MG – Brasil
marconi.medeiros@aracuai.mg.gov.br

***Abstract.** Digital forensics and incident response face critical challenges regarding scientific reproducibility and legal chain of custody compliance within virtualized and ephemeral environments. Traditional empirical approaches relying on manually filled post-test spreadsheets and ad-hoc provisioning scripts suffer from silent configuration drifts, loss of temporal causality, and vulnerability to forensic challenges in court. This paper introduces **DOML-F (DevOps Modeling Language for Forensic Laboratories)**, a declarative and observational model for the deployment, execution, evidence collection, and auditing of reproducible forensic experiments in Linux environments (Debian 13, Incus, BTRFS, and Ansible).*

***Resumo.** A computação forense e a resposta a incidentes enfrentam desafios críticos de reprodutibilidade científica e conformidade com a cadeia de custódia digital em ambientes virtualizados e efêmeros. Abordagens empíricas tradicionais baseadas em planilhas preenchidas manualmente pós-teste e scripts ad-hoc de provisionamento sofrem de desvios silenciosos de configuração, perda de causalidade temporal e vulnerabilidade a contestações periciais. Este artigo apresenta o **DOML-F (DevOps Modeling Language for Forensic Laboratories)**, um modelo declarativo e observacional para implantação, execução, coleta e auditoria de experimentos forenses replicáveis em nós Linux (Debian 13, Incus, BTRFS e Ansible).*

1. Introdução

A admissibilidade de evidências digitais no contexto pericial e judicial contemporâneo exige sólida fundamentação científica e rigor metodológico inquestionável. Padrões jurídicos e normativos internacionais — tais como o **Padrão Daubert** nos Estados Unidos, a norma **ISO/IEC 27037:2012** (*Guidelines for identification, collection, acquisition and preservation of digital evidence*) e o regramento de **Cadeia de Custódia** no Processo Penal brasileiro (Lei 13.964/2019, Arts. 158-A a 158-F) —

estabelecem que qualquer método pericial deve ser testável, submetido a escrutínio por pares, dotado de taxa de erro conhecida e plenamente **reprodutível** por terceiros independentes [Carrier 2003, Casey 2011].

Contudo, a prática de testes de laboratório em segurança e forense digital padece de fragilidades estruturais recorrentes:

I. Ambientes Artesanais e Não-Idempotentes: Alvos de teste são configurados manualmente ou por meio de sequências de comandos não versionadas, acumulando resíduos e artefatos de testes anteriores;

II. Fragilidade Probatória de Logs: Logs de sistema em texto plano (`/var/log/auth.log`) podem ser alterados retroativamente por usuários com privilégios de `root`, tornando vulnerável a comprovação de autoria e não-repúdio;

III. Dependência de Preenchimento Manual: Resultados de ensaios experimentais frequentemente são anotados após a execução em planilhas de cálculo isoladas, introduzindo erros de transcrição, viés humano e inconsistências entre o planejado e o executado;

IV. Anomalias Temporais e de Hardware: Saltos de tempo decorrentes de sincronização de relógio de parede (NTP) e instabilidades em osciladores de hardware (*clocksources*) causam distorções de cálculo de duração e quebram a ordem de eventos entre hosts distribuídos.

Para solucionar essas limitações, o projeto **DOML-F** propõe uma abordagem orientada a modelos (*Model-Driven Forensics*) combinada com princípios de Engenharia de Confiabilidade (*Site Reliability Engineering - SRE*) e Infraestrutura como Código (*IaC*). A premissa fundamental do DOML-F é a **separação estrita entre o Estado Desejado e o Estado Observado**:

- O **Estado Desejado** é expresso em uma especificação declarativa formal em YAML, validada por schemas computáveis antes de qualquer materialização;
- O **Estado Observado** é emitido de maneira concorrente e contínua como um fluxo estruturado de eventos imutáveis em JSON Lines (`.jsonl`), assinado digitalmente e auditado por um pipeline de validação criptográfica multicamadas;
- A **Projeção Analítica** (como planilhas ODS ou tabelas SQL) atua estritamente como visualização derivada dos eventos, eliminando qualquer edição manual direta como fonte de dados;
- **Portões Automáticos de Pré-Voo (*Preflight Gates*)**: Verificações estritas de pré-condições físicas e lógicas que impedem o início de experimentos caso a infraestrutura apresente desvios críticos.

Este artigo descreve a concepção, os fundamentos teóricos e a evolução histórica do DOML-F desde suas primeiras formulações (v0.1 e v0.2) até a maturidade arquitetural da v0.3. Demonstra-se como diagnósticos minuciosos de divergências e testes pré-operacionais na bancada física orientaram a construção de uma infraestrutura auditável baseada em *streams* encadeados, livro-razão de ingestão assíncrona (*ledger*), isolamento estrito de redes e preservação de cadeia de custódia segundo a ISO/IEC 27037.

2. Ambiente (background), Problema, objetivo e contribuição científica

A pergunta central é se a combinação de modelagem declarativa, automação e captura estruturada de observações permite implantar e avaliar um ambiente Linux de modo repetível e auditável, preservando a proveniência das evidências. A questão é operacional e verificável: não presume aceitação judicial nem superioridade universal sobre outros laboratórios.

O objetivo geral é desenvolver e avaliar um método de implantação automatizada e registro auditável para cenários controlados em Debian 13, Incus e BTRFS, materializados por Ansible.

Objetivos específicos:

- Descrever a topologia e as invariantes em DOML-F versionada;
- Automatizar C01 com testes funcionais;
- Registrar eventos e artefatos com IDs, hashes e rastreabilidade;
- Medir idempotência e reprodução sob critérios fixados;
- Analisar limitações e custo operacional da autenticação centralizada.

Contribuição metodológica imediata: o projeto especifica um contrato entre intenção, materialização e observação, estabelece como preservar versões e dados brutos e propõe critérios de invalidação. A contribuição empírica dependerá de C01 e dos cenários seguintes. A inspeção da matriz v0.2 é uma auditoria de instrumento, não uma medição de desempenho.

2.1 Modelagem Declarativa e DevOps em Segurança

Linguagens de modelagem declarativa em nuvem, tais como TOSCA (*Topology and Orchestration Specification for Cloud Applications*) [OASIS 2020] e a linguagem DOML original do projeto europeu PIACERE [Perez-Palacin et al. 2021], demonstraram o valor de separar a intenção de arquitetura da sua materialização por provedores como Terraform ou Ansible. Todavia, tais especificações foram concebidas exclusivamente para disponibilidade e entrega contínua (CI/CD), carecendo de construtos periciais nativos: não modelam hipóteses periciais, vetores de ataque controlados, *ground truth*, baselines imutáveis de integridade, cálculo estrito de desvio

temporal ou cadeias de custódia de evidências digitais. O DOML-F surge como uma especialização orientada à segurança e auditoria forense dessa linhagem declarativa.

2.2 Cadeia de Custódia Digital e ISO/IEC 27037:2012

A norma ISO/IEC 27037:2012 padroniza o tratamento de potenciais evidências digitais em quatro fases centrais: **Identificação, Coleta, Aquisição e Preservação** [ISO/IEC 2012]. Em ambientes periciais tradicionais, a conformidade baseia-se em formulários físicos de cadeia de custódia assinados por operadores. Em ambientes de nuvem e containers efêmeros, a automação dessa trilha exige mecanismos criptográficos que vinculem imediatamente o artefato adquirido ao seu hash, algoritmo, ferramenta coletora e instante UTC, impedindo contaminação posterior [Garfinkel 2010].

2.3 Criptografia e Canonicalização em JSON (JCS e RFC 8785)

Calcular hashes criptográficos sobre estruturas semiestruturadas como JSON sempre foi um desafio devido à não-ordenação nativa de chaves, variações de espaçamento em branco e representação de ponto flutuante [Bray 2014]. O padrão **JCS** (*JSON Canonicalization Scheme*, RFC 8785) padronizou a transformação determinística de representações JSON em sequências de bytes canônicas, viabilizando assinaturas digitais e cadeias de blocos verificáveis sem ambiguidade [Rundgren et al. 2020].

3. Evolução Histórica e Metodológica: Da v0.1 à v0.3

O amadurecimento do DOML-F foi impulsionado por um processo rigoroso de auditoria interna, diagnóstico de inconsistências e formulação de Decisões Metodológicas (DMs), conforme sintetizado na Tabela 1.

Tabela 1. Evolução metodológica e arquitetural do projeto DOML-F - v0.1 à v0.3.

Fase	Nome da Etapa	Itens & Especificações Técnicas
V01	Versão 0.1 (Concepção Inicial)	• A1: DOML-F SPEC v0.1 (YAML) – 5 Camadas Abstratas • A2: Matriz Experimental v0.1 (ODS) – Chaves Compostas (EXP + Execução) • A3: Preenchimento Manual Pós-Teste
	Diag Forense (2026-09-17)	• B1: Quebra Relacional Abas vs Dicionário (DIV-01) • B2: Inexistência de Cadeia de Custódia e Manifestos (DIV-03) • B3: Divergências = Injeção de Teste, não Drift • B4: Fragilidade de Edição Manual de Planilhas
V02	Versão 0.2 (Tentativa de Normalização)	• C1: Introdução de Execução_ID no Dicionário • C2: Cabeçalhos de Abas ainda Desnormalizados • C3: Persistência do Paradigma de Captura Manual
V03	Versão 0.3	• D1: Paradigma: JSONL Validado como Fonte Primária (DM-018)

Fase	Nome da Etapa	Itens & Especificações Técnicas
	(Maturidade Sistêmica Operacional)	• D2: Contrato de Eventos (15 Tipos, Envelopes Estritos) • D3: Integridade: Streams SHA-256 + Ledger + Ed25519 • D4: Relógios Duplos: UTC (<=1s) + Monotônicos • D5: 23 Entidades Relacionais (ISO 27037, Modelo Matriz v0.3) • D6: Projeção Declarativa Automática para ODS/SQL • D7: Portões de Pré-Voo G01-G12 e Preflight Operacional

3.1 Versão 0.1: Concepção Declarativa e Rupturas Estruturais

Na versão 0.1, concebeu-se a especificação formal `DOML-F-SPEC-v0.1.md` com 5 camadas (Meta, Application, Abstract Infrastructure, Concrete Infrastructure e Experimental/Forensic). Para acompanhar os testes, criou-se a planilha `matriz-experimental-v0.1.ods`.

Em setembro de 2026, uma auditoria estrutural exaustiva identificou graves divergências documentadas no *Relatório Técnico de Avaliação Forense e Divergências*:

- **Inconsistência de Chaves (DIV-01):** As abas de dados utilizavam chaves compostas desnormalizadas (EXP + Execução), enquanto o dicionário de dados começava a esboçar uma chave substituta (Execução_ID), quebrando a integridade referencial relacional;
- **Omissão da Cadeia de Custódia (DIV-03):** O modelo tabular possuía apenas uma coluna estática de “Integridade” na aba de evidências, omitindo as entidades essenciais de eventos de custódia e manifestos assinados;
- **Ambiguidade Semântica:** A aba “divergencias” registrava, na realidade, testes de tempo de detecção de ataques induzidos, e não o *drift* involuntário de infraestrutura;
- **Fragilidade Operacional:** Planilhas preenchidas manualmente violavam o princípio pericial de imutabilidade e auditabilidade temporal.

3.2 Versão 0.2: Tentativa Parcial de Normalização

A versão 0.2 introduziu a chave `Execução_ID` no dicionário de dados e tentou harmonizar os campos relacionais. Contudo, os cabeçalhos físicos das planilhas não foram refatorados, perpetuando conflitos entre a documentação e os artefatos ODS. Mais crítico ainda: a v0.2 continuava tratando a planilha ODS como a ferramenta de entrada de dados durante a execução do experimento, mantendo o risco de erros de digitação e ausência de trilha de auditoria em nível de milissegundos.

3.3 Versão 0.3: O Salto Paradigmático

A versão 0.3 foi estabelecida mediante a promulgação de 38 Decisões Metodológicas (DMs). A decisão mais marcante (**DM-018**) redefiniu a espinha dorsal do projeto: “A captura de dados durante a execução do experimento *NÃO* deve ocorrer diretamente em planilhas ODS. A fonte primária de verdade de todos os eventos é um fluxo contínuo em JSON Lines (.jsonl), validado em tempo real por JSON Schema. A planilha ODS torna-se exclusivamente uma projeção analítica gerada de forma determinística por exportadores automatizados.”

A v0.3 expandiu a modelagem para **23 entidades relacionais**, instituiu a trilha de custódia estrita com base nas 4 fases da ISO/IEC 27037, separou formalmente divergências (drift) de testes_deteccao (anomalias deliberadas), implementou cadeias de hashes locais, instituiu o livro-razão de ingestão assíncrona (*ledger*) e formalizou 12 portões de pré-voo automáticos (G01 a G12).

4. Arquitetura do Modelo DOML-F v0.3

A arquitetura do DOML-F v0.3 estrutura-se na interação coordenada entre a descrição declarativa do estado desejado, o contrato observacional de eventos e o motor de auditoria por camadas.

4.1 Contrato Observacional de Eventos

Cada evento ocorrido em qualquer componente do laboratório é serializado como uma linha independente em JSON Lines (`doml-f-event.schema.json`), contendo um envelope comum padronizado e um payload específico. O contrato define 15 tipos de eventos normativos, distribuídos em quatro domínios (Tabela 2).

Tabela 2: Tipos de eventos padronizados no Contrato de Eventos DOML-F v0.1.

Domínio	Tipo de Evento (event_type)	Emissor Primário	Destino na Matriz v0.3
Ciclo Metodológico	experiment.transitioned	Orquestrador	transicoes_experimento
Execução de Testes	execution.started	Orquestrador	execucoes
	execution.step.started	Ansible Callback	eventos_execucao
	execution.step.finished	Ansible Callback	eventos_execucao
	execution.completed	Orquestrador	execucoes
Evidências e Custódia	evidence.registered	Coletor Forense	evidencias
	custody.recorded	Coletor / Selador	custodia_eventos
	manifest.sealed	Selador Forense	manifestos e manifesto_evidencias
Validação e Métricas	validation.completed	Validador	validacoes_modelo / validacoes_execucao
	measurement.recorded	Agente de Medição	medicoes
	anomaly.recorded	Orquestrador / Analista	anomalias
	divergence.detected	Comparador de	divergencias

Domínio	Tipo de Evento (event_type)	Emissor Primário	Destino na Matriz v0.3
		Estado	
	detection_test.completed	Orquestrador	testes_deteccao
Infraestrutura / Integridade	ledger.event.accepted	Agregador Central	ledger_eventos
	record.corrected	Analista / Emissor	correcoes_registro

4.2 O Envelope Comum

O envelope normativo assegura a auditabilidade completa:

```
{
  "schema_version": "0.1.0",
  "event_id": "018f0000-0000-7000-8000-000000000001",
  "event_type": "execution.step.finished",
  "stream": {
    "stream_id": "STR-ANSIBLE-01",
    "emitter_type": "ansible_callback",
    "emitter_instance_id": "ansible-cb-host01",
    "host": "controller-01",
    "boot_id": "boot-uuid-xyz"
  },
  "stream_sequence": 2,
  "emitted_at_utc": "2026-09-20T12:00:06.250Z",
  "monotonic_ns": "7250000000",
  "causation_id": "018f0000-0000-7000-8000-000000000000",
  "context": {
    "exp": "EXP-001", "execucao_id": "EXEC-001",
    "doml_hash": "sha256:4a7b...f3c9", "commit": "git-sha-40chars"
  },
  "actor": {
    "ator_id": "ACT-SVC-ORCH", "principal": "orchestrator@LAB.LOCAL",
    "authentication_method": "KERBEROS_SSH"
  },
  "integrity": {
    "algorithm": "SHA-256",
    "previous_event_hash": "1111...1111",
    "event_hash": "2222...2222"
  },
  "payload": {
    "evento_id": "EVT-STEP-001", "resultado": "PASS",
    "fim_monotonico_ns": "7250000000", "duracao_ms": 5250,
    "intervencao": false
  }
}
```

4.3 Mecanismo Criptográfico de Integridade

A integridade no DOML-F é estabelecida em quatro níveis interdependentes:

- I. **Canonicalização e Hashing Determinístico (RFC 8785 + SHA-256):** Antes de computar o hash de qualquer evento, o documento JSON sofre cópia profunda e tem os campos autorreferenciais (integrity.event_hash e

integrity.signature) removidos. O conteúdo restante é processado via JCS (RFC 8785), produzindo uma representação em bytes determinística:

$$H_i = \text{SHA-256}(\text{JCS}(E_i \setminus \{\text{event_hash}, \text{signature}\}))$$

II. Encadeamento Hash-to-Hash por Stream Local: Cada processo emissor mantém uma cadeia linear própria indexada por `stream_sequence`. O evento inicial do stream possui `previous_event_hash = null`, enquanto cada evento subsequente embute o hash do evento anterior:

$$\text{integrity.previous_event_hash}_n = H_{n-1}$$

III. Livro-Razão de Ingestão Assíncrona (Ledger): Emissores distribuídos em diferentes hosts não possuem clock físico unificado nem sequência única. O agregador central no nó `foransi-host-00` emite eventos `ledger.event.accepted`, registrando a ordem contígua de recepção (`ledger_sequence`), conferindo e fixando os dados do evento de origem (`source_event_id`, `source_stream_id`, `source_event_hash`).

IV. Selamento e Assinatura Digital Ed25519: Ao final da execução, o emissor `manifest_sealer` emite `manifest.sealed`. O manifesto consolida o resumo de cada stream (primeira e última sequência, contagem total de eventos, primeiro e último hash), o hash final do ledger e a lista ordenada de todas as evidências registradas com seus respectivos hashes SHA-256. Uma assinatura digital assimétrica Ed25519 é calculada sobre os bytes canônicos do manifesto, assegurando autenticidade criptográfica e vínculo com a chave pública registrada.

4.4 Princípio de Imutabilidade e Retificações Auditáveis

O sistema implementa o princípio estrito de que **nenhum evento aceito pode ser alterado ou sobrescrito silenciosamente**. Caso um perito ou analista identifique que uma classificação foi inserida erroneamente (ex.: unidade de medida incorreta), ele é obrigado a emitir um evento do tipo `record.corrected`. Esse evento referencia o `target_event_id` e o `target_event_hash` exato da linha a ser retificada, indicando a justificativa (`correction_reason`) e o delta de campos (`corrected_fields`). O validador confirma a coincidência de hash antes de aceitar a retificação, preservando a história probatória intacta.

4.5 Controle Temporal Duplo e Imunidade a Saltos de Relógio

Para evitar falhas clássicas de correlação forense causadas por ajustes de passo (*clock steps*) do NTP ou desvios de fuso horário, o DOML-F adota um modelo de **tempo duplo**:

Correlação Global: Utiliza carimbos de tempo em UTC formatados segundo a ISO 8601 com sufixo Z (`emitted_at_utc`). Os nós mantêm sincronização contínua com `ntpsec`. O protocolo experimental estipula que qualquer desvio $|offset| > 1s$ antes do início bloqueia a execução, e durante o teste causa abortamento de emergência;

Cálculo de Durações Locais: A duração de etapas e comandos é calculada estritamente através de contadores do relógio monotônico do kernel (`CLOCK_MONOTONIC_RAW`) no host de controle:

$$\Delta t_{ms} = \left\lfloor \frac{t_{fim_monotonico_ns} - t_{inicio_monotonico_ns}}{1.000.000} \right\rfloor$$

Os valores de nanossegundos trafegam como strings decimais, contornando o limite de 53 bits de precisão inteira da especificação I-JSON (RFC 7493).

4.6 Motor de Projeção Relacional Declarativo

A matriz experimental v0.3 (planilha ODS e futuras tabelas SQL) é obtida através de um motor de projeção em Python (`projection.py`), regulado por um mapa declarativo em YAML (`event-projection-map-v0.2.yaml`). O motor navega nos eventos via JSON Pointer (RFC 6901), executa operações de ciclo de vida (`create`, `finalize`, `append`) e injeta perfis de proveniência (`source_event_id`, `source_stream_id`, `source_event_hash`, `projection_version`) em todas as linhas projetadas. Desse modo, qualquer célula da planilha ODS pode ser rastreada instantaneamente até o exato byte do JSONL que a gerou.

5. Implementação do Validador

O módulo `domlf_validator` foi concebido como um pipeline de inspeção e auditoria modular, implementado em Python 3.11+. Cada camada de verificação opera como um filtro estrito:

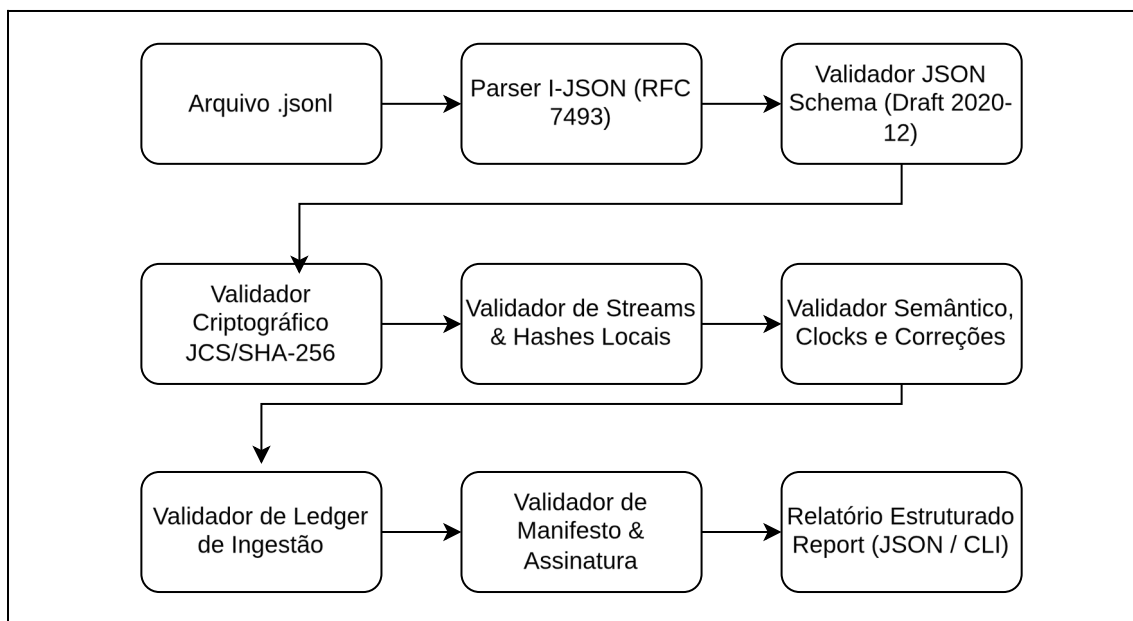


Figura 2: Pipeline de validação em camadas do domlf_validator.

- I. **Parser I-JSON** (`parser.py`): Inspecciona a sintaxe linha a linha com codificação UTF-8 estrita. Aciona hooks customizados (`_pairs`, `_int`, `_constant`) que barram chaves duplicadas no mesmo objeto, inteiros superiores a $2^{53}-1$ e constantes como NaN e Infinity;
- II. **Schema Validator** (`schema.py`): Valida a conformidade estrutural e de formatos (UUIDs, ISO 8601 UTC, expressões regulares de hashes SHA-256 de 64 caracteres hexadecimais);
- III. **Canonical Crypto** (`canonical.py`): Recalcula deterministicamente os hashes SHA-256 de todos os eventos via JCS e emite a issue `EVENT_HASH_MISMATCH` se houver divergência;
- IV. **Stream Analyzer** (`streams.py`): Confere se cada stream manteve sua identidade estável (`emitter_type`, `emitter_instance_id`, `host`, `boot_id`), se a sequência não possui omissões e se a cadeia de hashes anterior/atual está intacta;
- V. **Semantic Engine** (`semantic.py`): Verifica causalidade (`causation_id`), emparelha inícios e conclusões de tarefas Ansible, recalcula durações monotônicas e checka hashes de alvos em retificações imutáveis;
- VI. **Ledger Auditor** (`ledger.py`): Confere unicidade e correspondência exata entre as entradas do livro-razão e os eventos oficiais emitidos pelos nós;
- VII. **Manifest Verifier** (`manifest.py`): Confere se os limites declarados no manifesto batem com as contagens físicas dos streams e do ledger.

O comando resultante da interface de linha de comando (`cli.py`):

```
domlf-validate run-01.jsonl --schema schemas/doml-f-event.schema.json
--require-complete-ledger
```

retorna código de saída Unix 0 e diagnóstico detalhado em JSON somente se nenhuma violação de severidade ERROR for constatada.

6. Desenho Experimental, Portões de Pré-Voo e Achados da Bancada Física

6.1 Topologia do Ambiente Piloto

O ambiente físico de testes do DOML-F consiste em uma bancada pericial composta por quatro nós dedicados:

- **foransi-host-00 (Controlador Dedicado):** Executa o orquestrador Ansible, o repositório Git, o `domlf_validator`, os coletores centrais e o módulo de assinatura de manifestos Ed25519. Permanece isolado do conjunto avaliado;
- **foransi-host-01, 02 e 03 (Alvos Físicos Idênticos):** Equipados com processador AMD Athlon Gold 3150G (4 núcleos / 4 threads), ~15 GiB de RAM, SSD de 256 GB formatado em BTRFS (subvolume raiz em `/dev/sda2[/@rootfs]` com 225,22 GiB), executando Debian GNU/Linux 13.7 (kernel 6.12.107+deb13-amd64) e Incus 6.0.4.

A rede física opera no bloco `10.127.0.0/20`. Para evitar acoplamento a switches com suporte a VLAN física, cada host físico recebe uma sub-rede Incus exclusiva de classe /24 dentro de `10.179.0.0/20`:

- **host-01:** `10.179.1.0/24` (Hospeda `idp-01`: OpenLDAP, MIT Kerberos e SSH);
- **host-02:** `10.179.2.0/24` (Hospeda `files-01`: Samba e SSH);
- **host-03:** `10.179.3.0/24` (Hospeda `client-01`: Cliente de validação e coleta).

A comunicação inter-host é roteada explicitamente sobre a rede física e filtrada por regras determinísticas em `nftables`, garantindo a segregação estrita entre redes de gerência, ataque e evidências.

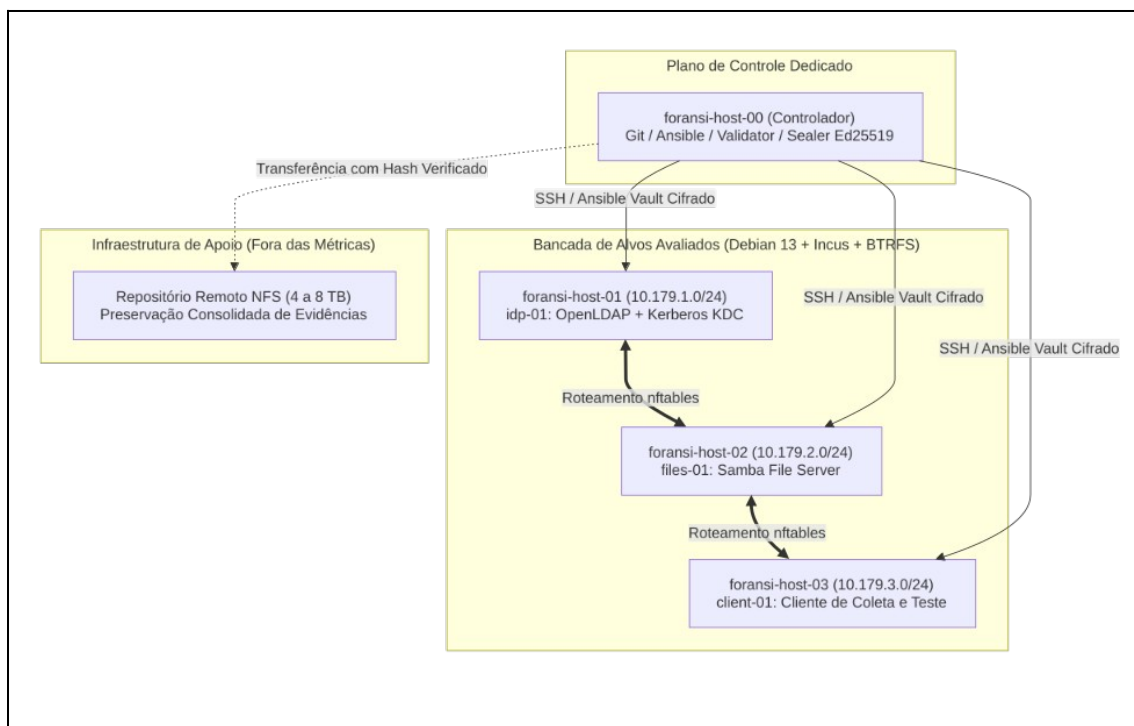


Figura 3: Topologia física e lógica de rede do laboratório experimental piloto.

6.2 Portões Automáticos de Pré-Voo (G01 a G12)

A arquitetura do DOML-F estipula que a execução de um experimento pericial só é autorizada se a infraestrutura for aprovada em 12 portões automáticos determinísticos (*preflight gates*), consolidados na Tabela 3.

Tabela 3: Portões automáticos de pré-voo (Preflight Gates) do DOML-F.

Portão	Nome do Portão	Critério Normativo de Aprovação	Condição de Bloqueio
G01	Identidade e Nomes	Nomes físicos corretos e FQDN reservado .test (RFC 6761)	Hostname divergente ou TLD não reservado
G02	Hardware e Versões	Debian 13.7, kernel 6.12, 4 vCPU, RAM ≥ 15 GiB, BTRFS e Incus 6.0.4	Disparidade de kernel ou pacotes de base
G03	Sincronização de Tempo	NTPsec contínuo, peer ativo, reach estável e $\text{offset} \leq 1,0$ s	Offset > 1 s, reach nulo ou saltos recorrentes
G04	Integridade do Git	Repositório estritamente limpo e commits registrados	Presença de arquivos <i>untracked</i> ou modificados
G05	Imagem de Contêiner	Mesma imagem Debian Incus com SHA-256 e fingerprint idênticos	Discrepância de imagem entre os nós
G06	Ausência Residual	Instâncias idp-01, files-01 e client-01 ausentes	Existência de contêineres remanescentes
G07	Rede e Roteamento	Sub-redes Incus /24 não sobrepostas, rotas e nftables ativos	Rota não autorizada para produção ou Internet
G08	Armazenamento Local	Subvolume BTRFS dedicado e ocupação operacional ≤ 150 GB	Uso próximo ao limite ou pool loop-backed
G09	Repositório Remoto	Montagem NFS acessível e teste de escrita/hash/leitura válido	Falha de conectividade ou divergência de hash
G10	Automação e Credenciais	SSH Ed25519 exclusivo, root remoto desativado e Vault funcional	Acesso por senha ativo ou falha de chave

Portão	Nome do Portão	Critério Normativo de Aprovação	Condição de Bloqueio
G11	Validação DOML-F	Modelo YAML validado sintática, semântica e forensemente	Inconsistência de esquema ou invariante
G12	Inicialização de Trilha	Emissão de IDs UUIDv7 e criação da estrutura EXP-xxxx/	Colisão de IDs ou falha de permissão

6.3 Achados Empíricos da Bancada Física: O Incidente PRE-C01-20260926-TIME

Durante a preparação operacional da execução exploratória do Cenário C01, realizada em 26 de setembro de 2026, os portões de pré-voo foram acionados sobre a bancada física. Esse ensaio gerou um dos achados mais significativos do projeto: a **deteção e isolamento de falhas de pré-condição antes da contaminação do experimento**.

6.3.1 Saneamento de Gerenciadores de Rede e Baseline

No preflight de rede, os inventários preliminares apontavam a presença de múltiplos arquivos de configuração nos alvos alocados. A execução do preflight revelou:

- O serviço ativo era exclusivamente `ifupdown` com a interface física `enp5s0` em `10.127.0.0/20`;
- Daemons como `NetworkManager`, `systemd-networkd` e `dhcpcd` estavam inativos; a presença de `/etc/dhcpcd.conf` era resíduo de pacotes desinstalados;
- Endereços legados da faixa `10.131.0.0/16` foram expurgados após reboot, restabelecendo a homogeneidade estrita dos alvos (`foransi-host-01`, `02` e `03`).

6.3.2 Falha de Sincronização em Rede Isolada (Portão G03)

Ao inspecionar o daemon de temporização `ntpsec` nos hosts alvos, constatou-se que a instalação padrão do Debian apontava exclusivamente para `*.debian.pool.ntp.org`. Como a bancada experimental é logicamente segregada da Internet para cumprir os requisitos forenses (INV-009 e RQ-41), as consultas externas falharam:

- `ntpq -pn` apresentou `reach = 0`, `stratum 16` e ausência de qualquer peer;
- A medição comparativa inicial via `ntpdate -q` contra o controlador revelou offsets inaceitáveis de `+3,576451 s` (`host-01`), `+3,369580 s` (`host-02`) e `+3,336984 s` (`host-03`), violando flagrantemente o teto de 1 segundo.

Refeita a reconfiguração dos clientes para utilizarem o controlador `foransi-host-00` (`10.127.0.10`) como servidor NTP local intermediário. Após a alteração, a medição instantânea pontual indicou offsets excelentes: `+0,013812 s` no `host-01`,

+0,010501 s no host-02 e +0,007536 s no host-03. Todavia, a inclusão redundante de diretivas `pool` e `server` em `/etc/ntpsec/ntp.conf` gerou associações concorrentes `unicast` e `.POOL.`, mantendo os clientes em estado candidato (+) sem estabilização definitiva do `system peer` (*).

6.3.3 Detecção de Instabilidade Grave de Hardware no Controlador

O achado mais crítico ocorreu no próprio nó controlador (`foransi-host-00`):

- O daemon `ntpsec` registrou saltos bruscos e recorrentes de relógio (*time stepped*): +0,485521 s às 12:14:05 e +0,496844 s às 12:30:26;
- Em 16 minutos foram observadas correções de relógio da ordem de 0,49 s, juntamente com `frequency = 75,677 ppm` e ausência de `system peer`. Esses dados caracterizam instabilidade temporal suficiente para invalidar o `preflight`, mas, isoladamente, não permitem atribuí-la a uma causa física específica, como falha do oscilador, nem estimar a deriva real do relógio a partir dos eventos de `time step`;
- O controlador entrou em estado anômalo: `leap_alarm`, `sync_unspec`, `no_sys_peer`, `stratum 16`, com instabilidade extrema de frequência (`frequency = 75,677 ppm`, `clk_wander = 13,33131`) e o arquivo `driftfile` estagnado em `0.000000`;
- Paralelamente, foram identificados erros de sintaxe no arquivo de configuração do NTPsec (opções obsoletas `nopeer` e `notrap` e conflito entre `disable monitor` e `limited`), além do uso inadequado de `tos orphan 5`.

6.3.4. Decisão Metodológica de Suspensão Imediata

Diante desses achados, o comitê pericial registrou o evento formal de suspensão:

PRE-C01-20260926-TIME

A preparação foi **imediatamente interrompida antes da criação de contêineres e antes do início do C01**. Metodologicamente, o evento foi classificado com estrita precisão: > **Classificação Normativa:** *Trata-se de uma falha de pré-condição da infraestrutura física/temporal, e NÃO de uma falha de implantação, idempotência ou reprodução do C01.*

Se o experimento tivesse prosseguido, os saltos observados no relógio de parede (`CLOCK_REALTIME`) poderiam comprometer a confiabilidade dos timestamps expressos em UTC, a correlação cronológica entre hosts e operações sensíveis à sincronização temporal, como a autenticação Kerberos. As durações locais calculadas por

CLOCK_MONOTONIC_RAW permaneceriam independentes desses ajustes do relógio de parede.

O plano de contingência aprovado estipulou:

1. Execução de teste A/B do clocksource do kernel Linux, comparando o comportamento do contador TSC (*Time Stamp Counter*) com o HPET (*High Precision Event Timer*);
2. Saneamento sintático das restrições do NTPsec e expurgo de declarações `tos orphan`;
3. Substituição do hardware dos computadores afetados caso a deriva persista;
4. **Critério Rígido de Desbloqueio do Portão G03:** Novo teste C01 só será liberado após comprovação de pelo menos 30 minutos contínuos sem *time stepped*, com peer selecionado, `reach` estável e offset absoluto $|\text{offset}| \leq 1,0$ s em todos os nós.

7. Discussão e Lições Aprendidas

7.1 A Inadequação das Planilhas como Instrumento de Captura Primária

A trajetória entre a v0.1 e a v0.3 forneceu uma lição pericial categórica: planilhas eletrônicas (sejam Excel ou Calc/ODS) são excelentes instrumentos de **inspeção visual e comunicação humana**, mas são falhas como repositórios de captura em tempo real. A incapacidade de impor regras de integridade transacional, tipagem rígida, encadeamento criptográfico e tolerância a concorrência assíncrona gerou as quebras de normalização diagnosticadas na v0.1. A adoção de streams JSONL imutáveis acoplados ao JCS (RFC 8785) se mostrou adequado neste momento para solucionar este problema.

7.2 Isolamento de Rede sem Complexidade de Hardware

A dependência inicial de switches com suporte a VLANs físicas e troncos 802.1Q foi identificada como um obstáculo à replicação do laboratório em diferentes centros periciais. A decisão metodológica de adotar sub-redes Incus /24 independentes por nó físico com roteamento ponto-a-ponto e filtragem rigorosa em `nftables` (**DM-034**) indicou que o isolamento pericial pode ser alcançado com uma única interface física por host, barateando a infraestrutura mantendo os requisitos para o experimento.

7.3 A Fragilidade da Temporização Física e a Eficácia do *Fail-Fast* Pericial

O incidente PRE-C01-20260926-TIME trouxe um achado metodológico relevante ao estudo: **não se pode presumir a integridade da camada temporal de software sem auditar a camada física de hardware**. A literatura frequentemente assume que

comandos como `ntpdate` ou daemons NTP em execução são suficientes para garantir correlação cronológica.

A experiência prática do DOML-F mostrou que falhas de sincronização temporal podem decorrer de diferentes camadas do sistema. Entre as hipóteses a serem discriminadas estão configurações inadequadas de `clocksource`, comportamento do oscilador físico, condições de hardware e configuração do próprio NTPsec. Redes segregadas sem uma referência temporal interna adequada constituem ainda uma condição operacional adicional de falha. A inclusão do portão automático G03 impediu que dados experimentalmente inválidos fossem produzidos: **é preferível abortar o experimento no pré-voo a registrar dados forenses sob uma base temporal corrompida.**

7.4 Delimitação de Responsabilidades: Trilha Experimental vs. Cadeia Legal

*O projeto assume explicitamente uma limitação de escopo: o DOML-F assegura uma **trilha de custódia experimental reproduzível**, alinhada aos requisitos técnicos de Identificação, Coleta, Aquisição e Preservação considerados à partir da ISO/IEC 27037:2012. Todavia, a conformidade jurídica integral com a cadeia de custódia processual (como a posse ininterrupta por perito oficial ou termo de apreensão policial) depende de atos administrativos externos, os quais a plataforma referencia por meio das identidades e papéis autenticados em `identidades_atores`.*

8. Conclusão e Trabalhos Futuros

O DOML-F apresenta uma nova proposta metodológica para a computação forense e a engenharia de segurança experimental. Ao integrar a expressividade de modelos declarativos (*IaC*) à robustez criptográfica de *streams* encadeados via JCS/SHA-256, livros-razão de ingestão assíncrona, portões automáticos de pré-voo e assinaturas digitais Ed25519, o sistema busca reduzir a dependência de preenchimento manual de dados e avança em direção à auditabilidade matemática ponta a ponta de experimentos forenses.

A evolução histórica documentada demonstra como o diagnóstico contínuo de divergências e a formalização de decisões metodológicas transformaram uma coleção inicial de scripts e tabelas em um ecossistema coerente de 23 entidades relacionais alinhadas à norma ISO/IEC 27037. A recente suspensão preventiva da execução exploratória do Cenário C01 pelo portão G03 demonstrou, na prática, a capacidade do modelo de bloquear uma execução diante de uma pré-condição temporal inválida.

Como trabalhos imediatos em andamento no roadmap técnico, destacam-se:

1. *Conclusão do teste comparativo de `clocksource` (`tsc` vs `hpet`) e saneamento do daemon NTPsec no controlador `foransi-host-00`;*

2. *Equalização dos manifests de pacotes dos nós alvos (foransi-host-01, 02 e 03) para expurgo de resíduos gráficos e multimídia;*

3. *Liberação e execução do piloto exploratório do Cenário C01 após comprovação de 30 minutos de estabilidade temporal contínua;*

4. *Execução das baterias experimentais pareadas ($N_0=5$ e $N \geq 10$) para quantificação definitiva do overhead do subsistema AAA (OpenLDAP + MIT Kerberos) sob condições estritamente controladas.*

9. Referências Bibliográficas

- Bray, T. (2014). *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 7159 / RFC 8259, Internet Engineering Task Force (IETF).
- Carrier, B. (2003). *Defining Digital Forensic Examination and Analysis Tools Using Abstraction Layers*. International Journal of Digital Evidence (IJDE), Volume 1, Edição 4
- Casey, E. (2011). *Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet*. Academic Press, 3rd edition.
- Garfinkel, S. L. (2010). *Digital forensics research: The next 10 years*. Digital Investigation, 7:S64–S73.
- ISO/IEC (2012). *ISO/IEC 27037:2012: Information technology — Security techniques — Guidelines for identification, collection, acquisition and preservation of digital evidence*. International Organization for Standardization, Geneva, Switzerland.
- ISO/IEC (2017). *ISO/IEC 17025:2017: General requirements for the competence of testing and calibration laboratories*. International Organization for Standardization, Geneva, Switzerland.
- Mills, D. L. (2010). *Computer Network Time Synchronization: The Network Time Protocol on Earth Space*. CRC Press, 2nd edition.
- NIST (2020). *Computer Forensic Tool Testing Program (CFTT)*. National Institute of Standards and Technology. Disponível em: <https://www.nist.gov/itl/csd/secure-systems-and-applications/computer-forensics-tool-testing-program-cftt>.
- OASIS (2020). *Topology and Orchestration Specification for Cloud Applications (TOSCA) Version 2.0*. OASIS Standard. Disponível em <https://docs.oasis-open.org/tosca/TOSCA/v2.0/TOSCA-v2.0.html>.
- Chiari, Michele & Di Nitto, Elisabetta & Mucientes, Adrián & Xiang, Bin. (2023). Developing a New DevOps Modelling Language to Support the Creation of Infrastructure as Code. 10.1007/978-3-031-23298-5_8.
- Rundgren, A., Jordan, B., e Erdtman, S. (2020). *JSON Canonicalization Scheme (JCS)*. RFC 8785, Internet Engineering Task Force (IETF). Disponível em <https://datatracker.ietf.org/doc/draft-rundgren-json-canonicalization-scheme/17/>
- Williams, N. (2015), Bray, T. (2015). *The I-JSON Message Format*. RFC 7493, Internet Engineering Task Force (IETF). Disponível em <https://datatracker.ietf.org/doc/rfc7493/>